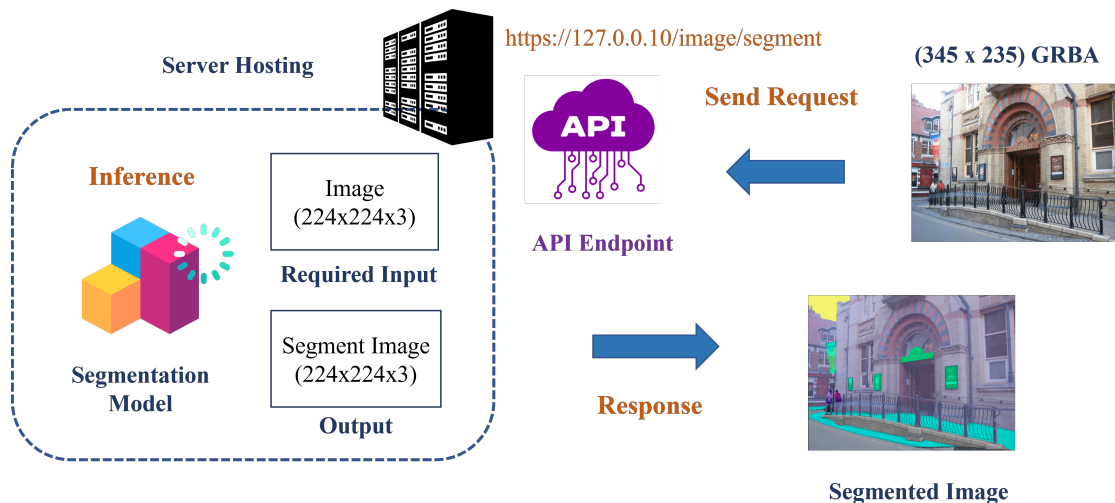


Triển khai mô hình Machine Learning với FastAPI

Nguyễn Đăng Nhã, Nguyễn Việt Anh Dương, Đinh Quang Vinh

I. Giới thiệu

Khi đối mặt với một vấn đề cần sử dụng AI hay cụ thể là Machine Learning để giải quyết, chúng ta biết quy trình để giải quyết vấn đề đó chính là xây dựng một mô hình ML phù hợp với yêu cầu của tác vụ đó, bao gồm: thu thập dữ liệu, xử lý dữ liệu, huấn luyện, đánh giá mô hình. Sau khi có được một mô hình đủ tốt, bước tiếp theo cần quan tâm đó là làm sao đưa mô hình này từ môi trường research sang môi trường production (như app mobile, website). Có một thuật ngữ được gọi là serving model, nghĩa là model sẽ được deploy và được xem như một web service, nhận đầu vào input và trả về output, hay nhận yêu cầu request rồi trả về kết quả response.



Hình 1: Quy trình deploy và serving mô hình segmentation qua API: server nhận ảnh đầu vào, xử lý bằng mô hình qua inference và trả về ảnh segment thông qua API endpoint.

Để thực hiện được điều đó, một trong những khái niệm không thể không nhắc tới đó là API. API (viết tắt là Application Programming Interface - Giao diện Lập trình Ứng dụng) là một tập hợp các quy tắc và công cụ cho phép các ứng dụng, hệ thống hoặc dịch vụ khác nhau giao tiếp và trao đổi dữ liệu với nhau. Deploy mô hình là quá trình mà chúng ta sẽ hosting mô hình

đó tại một server, triển khai các API endpoint để nhận các request (tức các input đầu vào), sau đó thay vì xử lý business logic như bình thường, thứ chúng ta cần là biến đổi dữ liệu đầu vào sao cho phù hợp với mô hình, đợi mô hình thực hiện tính toán (inference) rồi trả về output mong muốn.

Trong bài viết ngày hôm nay, chúng ta sẽ cùng nhau tìm hiểu về **FastAPI**, một framework Python hiện đại, mạnh mẽ, được thiết kế để xây dựng các API nhanh chóng, hiệu quả, và tiện lợi. FastAPI không chỉ nổi bật với hiệu năng cao nhờ xây dựng trên nền tảng ASGI (Asynchronous Server Gateway Interface) mà còn cung cấp các tính năng như tự động hóa tài liệu API hay validation dữ liệu với Pydantic. Trước khi đi vào ví dụ sử dụng FastAPI để triển khai mô hình Machine Learning, chúng ta sẽ thông qua một ví dụ thực tế: Student Management API - để minh họa cách xây dựng một ứng dụng FastAPI và để dễ dàng hiểu các khái niệm cơ bản nhất về framework này.

Mục lục

I.	Giới thiệu	1
II.	Các khái niệm cơ bản về FastAPI và ứng dụng quản lý sinh viên	4
II.1.	Routing, Include Route	4
II.2.	Giao thức HTTP	5
II.3.	Path parameter	6
II.4.	Pydantic và data model	7
II.5.	Tự động hóa tài liệu và Swagger UI	8
II.6.	Xây dựng CRUD đầy đủ cho ứng dụng	9
II.7.	Security, Middleware và mở rộng	10
III.	Triển khai mô hình với FastAPI	12
III.1.	Mô hình Object Detection	12
III.2.	Triển khai Project FastAPi chuẩn	17
III.3.	Thiết kế API Endpoint	18
III.4.	Schema Dữ liệu	20
III.5.	Flow Đọc hiểu Code	21
III.6.	Giao diện Gradio	21
IV.	Kết luận	22

II. Các khái niệm cơ bản về FastAPI và ứng dụng quản lý sinh viên

Sau đây, chúng ta sẽ từng bước tìm hiểu về các khái niệm của FastAPI và thực hiện triển khai code mẫu để hiểu hơn về nó.

II.1. Routing, Include Route

Trong FastAPI, **routing** là cơ chế cốt lõi để liên kết một đường dẫn URL cụ thể với một hàm Python xử lý yêu cầu. Về mặt lý thuyết, routing trong các web framework như FastAPI dựa trên nguyên tắc của *URL dispatching*, nơi server phân tích URL để xác định hành động cần thực hiện. FastAPI sử dụng thư viện Starlette làm nền tảng, hỗ trợ routing không đồng bộ (asynchronous) nhờ ASGI (Asynchronous Server Gateway Interface), giúp ứng dụng xử lý nhiều yêu cầu đồng thời mà không bị chặn. Điều này đặc biệt hữu ích trong các ứng dụng thời gian thực hoặc có lượng truy cập cao.

Khái niệm ✂

URL dispatching là quá trình ánh xạ URL tới hàm xử lý tương ứng, giống như một "mapping" dẫn server tới đúng chức năng cần thực thi.

Khi người dùng truy cập vào một đường dẫn, FastAPI sẽ biết cần thực thi hàm nào. Chúng ta sử dụng các *decorator* như `@app.get("/")`, `@app.post("/items/")` để định nghĩa các route này. Việc sử dụng decorator này không chỉ đơn giản hóa code mà còn tăng tính đọc hiểu, vì nó trực tiếp gắn kết phương thức HTTP với hàm xử lý, giảm thiểu lỗi cấu hình so với các framework như Flask nơi routing có thể phải khai báo thủ công hơn.

Khái niệm ✂

Decorator là một hàm bao quanh hàm khác để mở rộng chức năng mà không sửa đổi mã nguồn gốc, giống như "bọc" một hàm với logic bổ sung.

Khi ứng dụng trở nên phức tạp, việc tổ chức code là rất quan trọng. **APIRouter** giúp chúng ta nhóm các route liên quan vào một module riêng biệt. Điều này giúp code gọn gàng, dễ bảo trì và tái sử dụng. Về lý thuyết, APIRouter thúc đẩy nguyên tắc *modular design* trong phần mềm, nơi các thành phần được tách biệt theo trách nhiệm (Separation of Concerns - SoC). Theo tài liệu chính thức của FastAPI (<https://fastapi.tiangolo.com>), APIRouter cho phép định nghĩa prefix, tags, và dependencies chung cho một nhóm route, giúp tránh lặp code và dễ dàng tích hợp với các module khác. Trong thực tế, với các dự án lớn, chúng ta thường tách router theo domain (ví dụ: users, products), giúp team developer làm việc song song mà không xung đột.

Khái niệm ✂

Modular design là cách tổ chức code thành các module độc lập, mỗi module chịu trách nhiệm một chức năng riêng, giúp dễ bảo trì và mở rộng (thường mỗi module sẽ chứa 1 file `__init__.py`).

main.py

```
1 from fastapi import FastAPI
2 from .routers import students
3
4 app = FastAPI()
5 app.include_router(students.router)
6
7 @app.get("/")
8 def read_root():
9     return {"message": "Chào mừng đến với hệ thống quản lý sinh viên!"}
```

routers/students.py

```
1 from fastapi import APIRouter
2 from .models import Student
3
4 router = APIRouter(
5     prefix="/students",
6     tags=["Students"]
7 )
8
9 # Sample data
10 students_db = {
11     1: {"name": "Nguyễn Văn A", "age": 20, "is_active": True},
12     2: {"name": "Trần Thị B", "age": 22, "is_active": True}
13 }
14
15 @router.post("/", response_model=Student)
16 def create_student(student: Student):
17     student_id = len(students_db) + 1
18     students_db[student_id] = student
19     return student
20
21 @router.get("/")
22 def get_all_students():
23     return list(students_db.values())
```

II.2. Giao thức HTTP

Giao thức **HTTP** (Hypertext Transfer Protocol) là nền tảng của mọi giao tiếp trên web, được định nghĩa trong các RFC như RFC 7230-7235. Về lý thuyết, HTTP hoạt động theo mô hình client-server stateless, nghĩa là mỗi yêu cầu độc lập và không lưu trữ trạng thái giữa các request.

FastAPI hỗ trợ đầy đủ các phương thức HTTP chính, mỗi phương thức đại diện cho một loại hành động, tuân theo nguyên tắc RESTful (Representational State Transfer). REST nhấn mạnh vào việc sử dụng HTTP methods để thao tác tài nguyên, giúp API dễ mở rộng và dễ hiểu.

Khái niệm ✂

Stateless nghĩa là server không lưu thông tin về trạng thái của client giữa các request, mỗi request phải chứa đủ thông tin để xử lý.

FastAPI sử dụng ASGI để hỗ trợ HTTP/2 và HTTP/3, mang lại hiệu suất cao hơn so với HTTP/1.1 nhờ multiplexing (xử lý nhiều stream đồng thời) và header compression. Việc chọn đúng method HTTP giúp tránh các vấn đề bảo mật như idempotency (ví dụ: GET và DELETE nên an toàn khi lặp lại mà không thay đổi trạng thái). Các status code (như 200 OK, 404 Not Found) và headers (như Content-Type) cũng được FastAPI xử lý tự động, nhưng chúng ta cần chú ý tùy chỉnh để tránh tiết lộ thông tin không cần thiết.

Khái niệm ✂

Idempotent nghĩa là gọi cùng một request nhiều lần sẽ cho ra cùng một kết quả như lần đầu tiên, không tạo thêm thay đổi mới trên server.

- GET: Lấy dữ liệu từ server, nên là idempotent và safe.
- POST: Gửi dữ liệu mới để tạo tài nguyên, không idempotent.
- PUT: Cập nhật toàn bộ một tài nguyên, idempotent.
- PATCH: Cập nhật một phần của tài nguyên, không nhất thiết idempotent.
- DELETE: Xóa một tài nguyên, idempotent.

Theo <https://fastapi.tiangolo.com>, FastAPI khuyến khích sử dụng `async def` cho các route để tận dụng đầy đủ lợi thế của HTTP không đồng bộ, đặc biệt trong I/O-bound tasks như gọi database.

Khái niệm ✂

I/O-bound task (nhiệm vụ bị ràng buộc bởi I/O) là loại công việc mà thời gian thực hiện chủ yếu bị chi phối bởi việc nhập/xuất dữ liệu (Input/Output), chứ không phải do CPU tính toán.

II.3. Path parameter

Path parameter là một phần của URL, được sử dụng để xác định một tài nguyên cụ thể, dựa trên nguyên tắc URI (Uniform Resource Identifier) trong HTTP. FastAPI sử dụng cú pháp `{tên_tham_số}` để định nghĩa path parameter, và tận dụng type hinting của Python để tự động validate dữ liệu. Về lý thuyết, điều này áp dụng khái niệm *type safety* từ lập trình chức năng, giúp phát hiện lỗi sớm tại runtime thay vì deep trong code.

Khái niệm ✂

Type safety là cơ chế đảm bảo dữ liệu đúng kiểu (int, str, v.v.) trước khi xử lý, giúp giảm lỗi runtime và tăng độ tin cậy của code.

Ví dụ, để lấy thông tin của một sinh viên có ID cụ thể: FastAPI tự động xác thực kiểu dữ liệu (ở đây là int) và trả về lỗi nếu dữ liệu không hợp lệ, như 422 Unprocessable Entity. Ở đây, chúng ta thường nên thêm constraints như enum hoặc regex (biểu thức chính quy) (ví dụ: `student_id: int = Path(..., ge=1)`) để hạn chế input xấu, giảm tải cho database và tránh injection attacks. Tài liệu FastAPI gợi ý sử dụng Path từ fastapi để tùy chỉnh description, deprecated, v.v., giúp docs API chi tiết hơn.

Khái niệm ✂

Injection Attack là khi kẻ xấu cố tình chèn mã (như SQL) vào input để thao túng hệ thống, ví dụ: nhập lệnh xóa dữ liệu thay vì tên.

routers/students.py

```

1 from fastapi import Path
2 from .models import Student
3
4 @router.get("/{student_id}", response_model=Student)
5 def get_student(student_id: int = Path(..., ge=1)):
6     student = students_db.get(student_id)
7     if not student:
8         raise HTTPException(status_code=404, detail="Không tìm thấy sinh viên")
9     return student

```

II.4. Pydantic và data model

Pydantic là thư viện được FastAPI tích hợp sẵn để xác thực và xử lý dữ liệu, dựa trên Python type hints và dataclass. Về lý thuyết, Pydantic áp dụng **data validation** và **serialization / deserialization** để đảm bảo dữ liệu sạch, tuân thủ schema. Điều này giống như JSON Schema nhưng tích hợp sâu với Python, giúp tránh lỗi như type mismatch hoặc missing fields.

Khái niệm ✂

Serialization/Deserialization là quá trình chuyển đổi dữ liệu thành định dạng có thể lưu trữ/gửi (serialization) và ngược lại (deserialization).

Bằng cách định nghĩa các **model** kế thừa từ `pydantic.BaseModel`, chúng ta có thể đảm bảo dữ liệu đầu vào và đầu ra tuân thủ một cấu trúc nhất định. Nested models (như StudentDetails) hỗ trợ dữ liệu phức tạp, và validators tùy chỉnh (ví dụ: `@validator`) cho phép logic business cụ thể. Pydantic giúp loại bỏ sớm các bug bằng cách validate sớm, đặc biệt trong microservices nơi dữ liệu di chuyển giữa các service. Ngoài ra, nó hỗ trợ ORM integration với SQLAlchemy, giúp map model trực tiếp với database schema.

models.py

```

1 from pydantic import BaseModel, validator
2
3 class Student(BaseModel):
4     name: str
5     age: int
6     is_active: bool = True
7
8     @validator('age')
9     def age_must_be_positive(cls, v):
10         if v < 0:
11             raise ValueError('Age must be positive')
12         return v
13
14 class Course(BaseModel):
15     course_name: str
16     credits: int
17
18 class StudentDetails(BaseModel):
19     student_info: Student
20     courses: list[Course]

```

II.5. Tự động hóa tài liệu và Swagger UI

Một trong những điểm mạnh nhất của FastAPI là khả năng tự động tạo tài liệu API tương tác dựa trên OpenAPI standard (<https://swagger.io/specification/>). Về lý thuyết, OpenAPI định nghĩa schema cho API, bao gồm paths, parameters, responses, giúp tool như Swagger UI render giao diện. Chỉ cần chạy ứng dụng và truy cập `/docs`, bạn sẽ thấy giao diện **Swagger UI**, cho phép bạn xem và thử nghiệm các endpoint một cách trực quan. Ngoài ra, `/redoc` cung cấp giao diện khác với docs.

Khái niệm ✂

OpenAPI là chuẩn để mô tả cấu trúc API, giúp tự động tạo tài liệu và giao diện thử nghiệm, giảm công sức viết tài liệu thủ công.

Tính năng này tiết kiệm hàng giờ so với viết docs thủ công như ở Django, và dễ tùy chỉnh bằng `response_model`, `summary`, `description`, và `operation_id` trong decorator. Ví dụ, bạn có thể thêm `operation_id="get_student_by_id"` để đặt tên endpoint trong Swagger UI, hoặc `summary="Lấy thông tin sinh viên"` để mô tả ngắn gọn. Để tùy chỉnh sâu hơn, FastAPI cho phép override schema OpenAPI bằng cách định nghĩa hàm `custom_openapi` trong ứng dụng, hữu ích khi cần thêm metadata hoặc ẩn một số endpoint khỏi tài liệu.

custom_openapi.py

```

1 from fastapi import FastAPI
2 from fastapi.openapi.utils import get_openapi
3

```



```

4 app = FastAPI()
5
6 def custom_openapi():
7     if app.openapi_schema:
8         return app.openapi_schema
9     openapi_schema = get_openapi(
10         title="Hệ thống quản lý sinh viên",
11         version="1.0.0",
12         description="API quản lý sinh viên với FastAPI",
13         routes=app.routes,
14     )
15     openapi_schema["info"]["x-logo"] = {
16         "url": "https://fastapi.tiangolo.com/img/logo-margin/logo-teal.png"
17     }
18     app.openapi_schema = openapi_schema
19     return app.openapi_schema
20
21 app.openapi = custom_openapi

```

II.6. Xây dựng CRUD đầy đủ cho ứng dụng

CRUD là viết tắt của Create, Read, Update, Delete, đại diện cho các hoạt động cơ bản trên dữ liệu bền vững (persistent data). Về lý thuyết, CRUD map trực tiếp với HTTP methods trong REST, đảm bảo API idempotent và stateless. FastAPI làm cho việc implement CRUD dễ dàng nhờ dependency injection và model validation.

Khái niệm ✂

REST (Representational State Transfer) là phong cách kiến trúc API sử dụng HTTP methods để thao tác tài nguyên, đảm bảo tính mở rộng và đơn giản.

Dưới đây là ví dụ đầy đủ, sử dụng POST để tạo dữ liệu mẫu và các route khác xử lý dữ liệu nhất quán. Chúng ta sử dụng `response_model` để đảm bảo output đúng schema và thêm error handling để graceful degradation.

routers/students.py

```

1 from fastapi import HTTPException, APIRouter
2 from .models import Student
3
4 router = APIRouter(prefix="/students", tags=["Students"])
5
6 students_db = {
7     1: Student(name="Nguyễn Văn A", age=20, is_active=True),
8     2: Student(name="Trần Thị B", age=22, is_active=True)
9 }
10
11 # C (Create)
12 @router.post("/", response_model=Student)

```

```

13 def create_student(student: Student):
14     student_id = len(students_db) + 1
15     students_db[student_id] = student
16     return student
17
18 # R (Read)
19 @router.get("/", response_model=list[Student])
20 def read_students():
21     return list(students_db.values())
22
23 @router.get("/{student_id}", response_model=Student)
24 def read_student(student_id: int):
25     student = students_db.get(student_id)
26     if not student:
27         raise HTTPException(status_code=404, detail="Không tìm thấy sinh viên")
28     return student
29
30 # U (Update) - PUT for full update
31 @router.put("/{student_id}", response_model=Student)
32 def update_student(student_id: int, student: Student):
33     if student_id not in students_db:
34         raise HTTPException(status_code=404, detail="Không tìm thấy sinh viên")
35     students_db[student_id] = student
36     return student
37
38 # U (Update) - PATCH for partial update
39 @router.patch("/{student_id}", response_model=Student)
40 def partial_update_student(student_id: int, student_update: dict):
41     if student_id not in students_db:
42         raise HTTPException(status_code=404, detail="Không tìm thấy sinh viên")
43     current_student = students_db[student_id].dict()
44     current_student.update(student_update)
45     students_db[student_id] = Student(**current_student)
46     return students_db[student_id]
47
48 # D (Delete)
49 @router.delete("/{student_id}")
50 def delete_student(student_id: int):
51     if student_id not in students_db:
52         raise HTTPException(status_code=404, detail="Không tìm thấy sinh viên")
53     del students_db[student_id]
54     return {"message": "Sinh viên đã được xóa"}

```

II.7. Security, Middleware và mở rộng

Security: FastAPI hỗ trợ bảo mật API bằng cách sử dụng các Dependency, dựa trên dependency injection pattern để reusable code. Về lý thuyết, security schemes như OAuth2, API keys tuân theo OWASP guidelines để chống lại threats như unauthorized access. Tích hợp JWT (JSON Web Tokens) với libraries như python-jose giúp secure API mà không lưu session, lý tưởng cho scalable apps.

Khái niệm ✂

Dependency Injection là kỹ thuật cung cấp các phụ thuộc (như hàm xác thực) cho một hàm khác, giúp code linh hoạt và dễ kiểm thử.

security_example.py

```

1 from fastapi import Depends, HTTPException
2 from fastapi.security import OAuth2PasswordBearer
3 from typing import Annotated
4
5 oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")
6
7 def is_admin(token: Annotated[str, Depends(oauth2_scheme)]):
8     if token != "admin_secret_token":
9         raise HTTPException(status_code=401, detail="Unauthorized")
10
11 @router.delete("/{student_id}", dependencies=[Depends(is_admin)])
12 def delete_student_secure(student_id: int):
13     return {"message": "Sinh viên đã được xóa bởi admin"}

```

Đoạn code minh họa cách bảo vệ API bằng **token** trong FastAPI.

- `OAuth2PasswordBearer(tokenUrl="token")` - Đây là cách FastAPI yêu cầu client gửi kèm **token** trong request. Thông thường, token được gửi trong phần **Authorization Header** dưới dạng: `Authorization: Bearer <token>`
- `def is_admin(...)` - Hàm này là một **dependency**, dùng để kiểm tra token mà người dùng gửi lên. Nếu token khác `"admin_secret_token"` → trả về lỗi **401 Unauthorized**. Nếu đúng → cho phép tiếp tục.
- `@router.delete("/student_id", dependencies=[Depends(is_admin)])` - API này định nghĩa một **endpoint DELETE**, dùng để xóa sinh viên theo ID. Nhưng trước khi chạy, FastAPI sẽ gọi `is_admin()` để đảm bảo chỉ **admin** mới có quyền.

Đơn giản hóa: Khi bạn muốn gọi API xóa sinh viên, bạn phải đưa ra “chìa khoá bí mật” (token admin). Nếu không có chìa khoá đúng, bạn sẽ bị từ chối ngay.

Middleware: Middleware là các hàm chạy trước và sau mỗi yêu cầu, dựa trên chain of responsibility pattern. Nó hữu ích cho các tác vụ như xử lý CORS (Cross-Origin Resource Sharing), logging hoặc nén dữ liệu. Về lý thuyết, middleware giúp tách biệt cross-cutting concerns khỏi business logic. Tài liệu FastAPI gợi ý custom middleware cho advanced use như rate limiting với slowapi.

Khái niệm ✂

Chain of Responsibility là mẫu thiết kế nơi nhiều đối tượng xử lý một yêu cầu theo chuỗi, mỗi đối tượng quyết định tiếp tục hay dừng.

cors_middleware.py

```

1 from fastapi.middleware.cors import CORSMiddleware
2
3 app.add_middleware(
4     CORSMiddleware,
5     allow_origins=["*"],
6     allow_credentials=True,
7     allow_methods=["*"],
8     allow_headers=["*"],
9 )

```

- `allow_origins=["*"]` Cho phép mọi domain (nguồn) đều có thể gửi request đến API. (Trong thực tế nên giới hạn cụ thể, ví dụ: `["https://myapp.com"]`).
- `allow_credentials=True` Cho phép gửi kèm thông tin nhạy cảm như cookie, Authorization header.
- `allow_methods=["*"]` Cho phép tất cả các phương thức HTTP (GET, POST, PUT, DELETE, ...).
- `allow_headers=["*"]` Cho phép mọi loại header trong request.

Đơn giản hóa: Middleware này giống như “cửa bảo vệ”, quyết định xem trình duyệt từ đâu được phép gọi API. Ở đây thì mở cửa cho tất cả, nên tiện cho phát triển nhưng kém an toàn khi triển khai thật.

III. Triển khai mô hình với FastAPI

III.1. Mô hình Object Detection

Ở phần này, chúng ta sẽ chọn một mô hình Machine learning, Deep Learning bất kỳ. Ở bài viết này, chúng ta sẽ sử dụng mô hình Object Detection được public ở link HuggingFace trên [yainage90/fashion-object-detection](#). Khi triển khai bất cứ mô hình nào, chúng ta đều cần đọc và hiểu rõ định dạng Input của mô hình này sử dụng.

Code mẫu quá trình inference của mô hình

```

1 from PIL import Image
2 import torch
3 from transformers import AutoImageProcessor, AutoModelForObjectDetection
4
5 device = 'cpu'
6 if torch.cuda.is_available():
7     device = torch.device('cuda')
8 elif torch.backends.mps.is_available():
9     device = torch.device('mps')
10

```

```

11 ckpt = 'yainage90/fashion-object-detection'
12 image_processor = AutoImageProcessor.from_pretrained(ckpt)
13 model = AutoModelForObjectDetection.from_pretrained(ckpt).to(device)
14
15 image = Image.open('<path/to/image>').convert('RGB')
16
17 with torch.no_grad():
18     inputs = image_processor(images=[image], return_tensors="pt")
19     outputs = model(**inputs.to(device))
20     target_sizes = torch.tensor([[image.size[1], image.size[0]]])
21     results = image_processor.post_process_object_detection(outputs, threshold=0.4,
22                                                             target_sizes=target_sizes)[0]
23
24     items = []
25     for score, label, box in zip(results["scores"], results["labels"], results["boxes"]):
26
27         score = score.item()
28         label = label.item()
29         box = [i.item() for i in box]
30         print(f"{model.config.id2label[label]}: {round(score, 3)} at {box}")
31         items.append((score, label, box))

```

Thông qua code mẫu, chúng ta biết chỉ cần convert Image sang RGB channel và sử dụng image-processor để xử lý ảnh trước khi đưa vào mô hình Object Detection chính (và mô hình này tất nhiên đã được pretrained). Qua đó, để thực hiện khởi tạo và detect ảnh, chúng ta sẽ triển khai 2 services như sau:

model_service.py

```

1 import torch
2 from transformers import AutoImageProcessor, AutoModelForObjectDetection
3 from PIL import Image
4 from typing import List, Dict, Any
5 import time
6
7 from app.core.config import settings
8 from app.utils.logger import logger
9
10 class ModelService:
11     def __init__(self):
12         self.device = self._get_device()
13         self.image_processor = None
14         self.model = None
15         self.load_model()
16
17     def _get_device(self) -> torch.device:
18         """Determine the best available device"""
19         if torch.cuda.is_available():
20             device = torch.device('cuda')
21             logger.info("Using CUDA device")
22         elif torch.backends.mps.is_available():
23             device = torch.device('mps')

```

```

24         logger.info("Using MPS device")
25     else:
26         device = torch.device('cpu')
27         logger.info("Using CPU device")
28     return device
29
30 def load_model(self):
31     """Load the model and processor"""
32     try:
33         logger.info(f"Loading model from {settings.MODEL_CHECKPOINT}")
34         self.image_processor = AutoImageProcessor.from_pretrained(settings.
35                                                                     MODEL_CHECKPOINT)
36         self.model = AutoModelForObjectDetection.from_pretrained(settings.
37                                                                     MODEL_CHECKPOINT).to(self.device)
38         self.model.eval()
39         logger.info("Model loaded successfully")
40     except Exception as e:
41         logger.error(f"Error loading model: {str(e)}")
42         raise
43
44 def preprocess_image(self, image: Image.Image) -> Dict[str, torch.Tensor]:
45     """Preprocess image for model input"""
46     return self.image_processor(images=[image], return_tensors="pt")
47
48 def postprocess_detections(self, outputs, target_sizes, threshold: float) -> List[
49     Dict[str, Any]]:
50     """Postprocess model outputs into readable format"""
51     results = self.image_processor.post_process_object_detection(
52         outputs, threshold=threshold, target_sizes=target_sizes
53     )[0]
54
55     detections = []
56     for score, label, box in zip(results["scores"], results["labels"], results["
57         boxes"]):
58         detection = {
59             "label": self.model.config.id2label[label.item()],
60             "score": round(score.item(), 4),
61             "bounding_box": {
62                 "xmin": round(box[0].item(), 2),
63                 "ymin": round(box[1].item(), 2),
64                 "xmax": round(box[2].item(), 2),
65                 "ymax": round(box[3].item(), 2)
66             }
67         }
68         detections.append(detection)
69
70     return detections
71
72 def detect_objects(self, image: Image.Image, threshold: float = None) -> Dict[str,
73     Any]:
74     """Main detection method"""
75     if threshold is None:
76         threshold = settings.DETECTION_THRESHOLD

```

```

73     start_time = time.time()
74
75     with torch.no_grad():
76         inputs = self.preprocess_image(image)
77         inputs = {k: v.to(self.device) for k, v in inputs.items()}
78
79         outputs = self.model(**inputs)
80         target_sizes = torch.tensor([[image.size[1], image.size[0]]]).to(self.
                                     device)
81
82         detections = self.postprocess_detections(outputs, target_sizes, threshold)
83         processing_time = time.time() - start_time
84
85     return {
86         "detections": detections,
87         "processing_time": processing_time,
88         "image_size": {"width": image.size[0], "height": image.size[1]}
89     }
90
91 # Global model service instance
92 model_service = ModelService()

```

detect_service.py

```

1  from PIL import Image
2  import io
3  from typing import List, Dict, Any
4
5  from app.services.model_service import model_service
6  from app.models.responses import DetectionResponse, ErrorResponse
7  from app.utils.image_processor import image_processor
8  from app.utils.logger import logger
9
10 class DetectionService:
11     @staticmethod
12     def detect_from_bytes(image_bytes: bytes, threshold: float = None) ->
                                     DetectionResponse:
13         """Detect objects from image bytes"""
14         try:
15             # Validate image
16             if not image_processor.validate_image(image_bytes):
17                 return ErrorResponse(
18                     success=False,
19                     message="Invalid image file",
20                     error_code="INVALID_IMAGE",
21                     details={"file_type": "Unable to determine image format"}
22                 )
23
24             # Convert to RGB PIL Image
25             image = image_processor.convert_to_rgb(image_bytes)
26             image_info = image_processor.get_image_info(image)
27

```

```

28     # Detect objects
29     result = model_service.detect_objects(image, threshold)
30
31     return DetectionResponse(
32         success=True,
33         message="Detection completed successfully",
34         detections=result["detections"],
35         processing_time=round(result["processing_time"], 4),
36         image_size=result["image_size"],
37         total_detections=len(result["detections"])
38     )
39
40     except Exception as e:
41         logger.error(f"Error in detection from bytes: {str(e)}", exc_info=True)
42         return ErrorResponse(
43             success=False,
44             message="Failed to process image",
45             error_code="PROCESSING_ERROR",
46             details={"error": str(e)},
47             stack_trace=str(e) if logger.level == 10 else None # Only include
                                                                stack trace in debug
48         )
49
50     @staticmethod
51     def detect_from_pil(image: Image.Image, threshold: float = None) ->
        DetectionResponse:
52         """Detect objects from PIL Image"""
53         try:
54             image_info = image_processor.get_image_info(image)
55
56             # Detect objects
57             result = model_service.detect_objects(image, threshold)
58
59             return DetectionResponse(
60                 success=True,
61                 message="Detection completed successfully",
62                 detections=result["detections"],
63                 processing_time=round(result["processing_time"], 4),
64                 image_size=result["image_size"],
65                 total_detections=len(result["detections"])
66             )
67
68         except Exception as e:
69             logger.error(f"Error in detection from PIL: {str(e)}", exc_info=True)
70             return ErrorResponse(
71                 success=False,
72                 message="Failed to process image",
73                 error_code="PROCESSING_ERROR",
74                 details={"error": str(e)}
75             )
76
77     @staticmethod
78     def get_annotated_image(image: Image.Image, detections: List[Dict[str, Any]]) ->
        Image.Image:

```



```

79     """Get image with bounding boxes drawn"""
80     return image_processor.draw_bounding_boxes(image, detections)

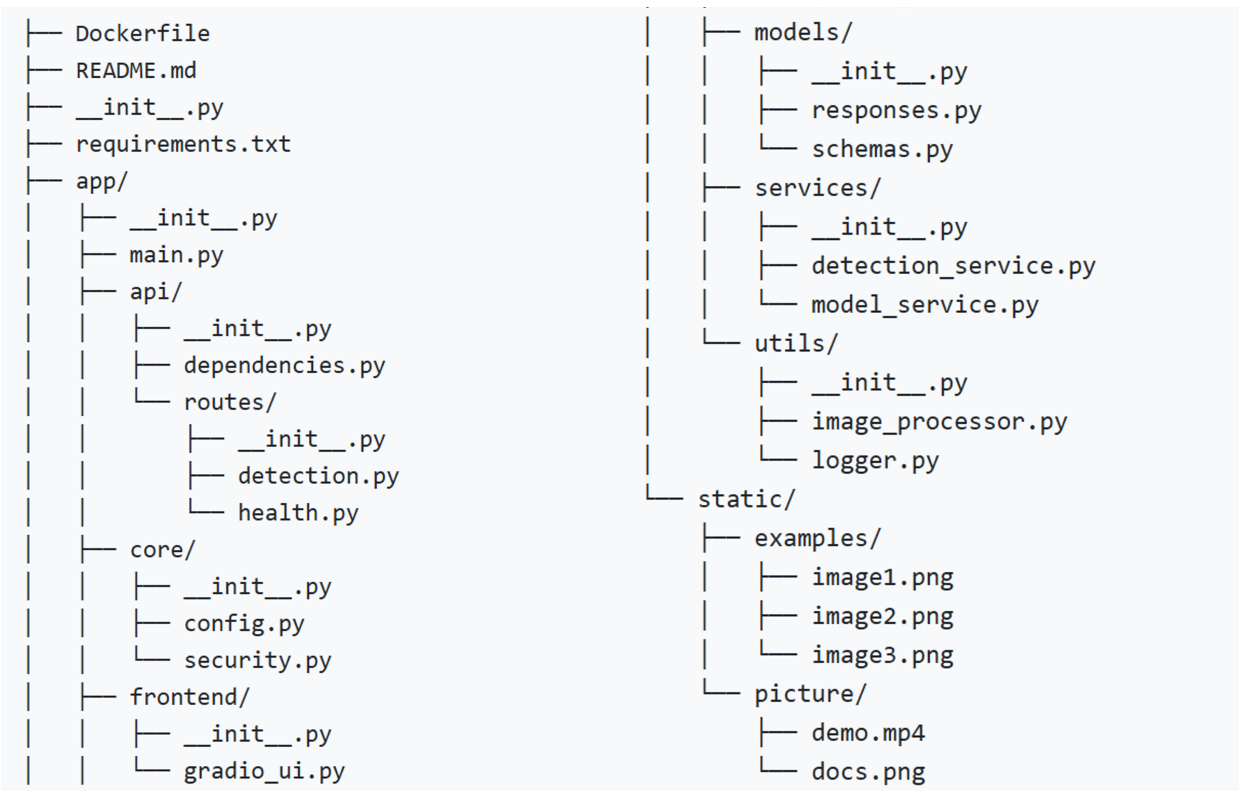
```

III.2. Triển khai Project FastAPI chuẩn

Project **fashion object detection** được upload tại **fashion-detection-app**. Chúng ta sẽ cùng nhau phân tích cấu trúc repository và flow đọc hiểu code.

Hiện tại, mô hình đã được triển khai sẵn sàng để xử lý input đầu vào theo yêu cầu. Do đó, trọng tâm của chúng ta là thiết kế API endpoint để nhận ảnh, xử lý, và trả về kết quả. Đối với API endpoint, chúng ta cần thiết kế một Route để nhận ảnh thông qua request, đồng thời quy định rõ ràng về schema mà hệ thống sẽ nhận và trả về. Dưới đây là phân tích chi tiết về cấu trúc repository và cách tổ chức code.

Tổ chức Thư mục



Hình 2: Cấu trúc project triển khai ML/DL Model bằng FastAPI

Cấu trúc thư mục của project được thiết kế theo mô hình modular, tuân thủ các tiêu chuẩn của một ứng dụng FastAPI, giúp dễ dàng mở rộng và bảo trì. Cụ thể:

- **app/**: Thư mục chính chứa toàn bộ mã nguồn ứng dụng.
 - **api/**: Chứa các định nghĩa route và endpoint của FastAPI. Ví dụ, một endpoint `/detect` sẽ được định nghĩa để nhận ảnh và trả về kết quả phát hiện.

- **core/**: Chứa các cấu hình (như biến môi trường) và logic bảo mật (như xác thực JWT hoặc API key).
 - **models/**: Chứa các schema Pydantic để validate dữ liệu đầu vào (request) và đầu ra (response), đảm bảo dữ liệu đúng định dạng.
 - **services/**: Chứa logic mô hình, bao gồm tải mô hình Hugging Face (yainage90/fashion-object-detection), xử lý ảnh, và chạy inference.
 - **utils/**: Các hàm tiện ích hỗ trợ, như xử lý file ảnh hoặc logging.
 - **frontend/**: Giao diện người dùng được xây dựng bằng Gradio, cho phép upload ảnh và hiển thị kết quả trực quan.
 - **main.py**: Điểm khởi đầu của ứng dụng FastAPI, nơi khởi tạo ứng dụng, include routes, và thiết lập middleware.
- **requirements.txt**: Liệt kê các thư viện Python cần thiết, như `fastapi`, `uvicorn`, `transformers`, `pydantic`, v.v.
 - **Dockerfile**: Cấu hình để đóng gói ứng dụng vào container Docker, hỗ trợ triển khai production.
 - **README.md**: Tài liệu hướng dẫn cách cài đặt, chạy, và sử dụng project.

Cấu trúc này đảm bảo tính tách biệt (separation of concerns), cho phép phát triển độc lập từng module. Ví dụ, có thể cập nhật mô hình trong `services/` mà không ảnh hưởng đến giao diện trong `frontend/`.

III.3. Thiết kế API Endpoint

API endpoint là thành phần cốt lõi để nhận và xử lý ảnh đầu vào. Trong project này, một endpoint chính (ví dụ: `POST /detect`) được thiết kế để:

- **Nhận input**: Người dùng upload một file ảnh (định dạng JPEG, PNG, v.v.) thông qua request HTTP.
- **Validate input**: Sử dụng schema Pydantic trong `models/` để kiểm tra tính hợp lệ của dữ liệu đầu vào (ví dụ: kiểm tra định dạng file, kích thước ảnh).
- **Xử lý ảnh**: Gọi các hàm từ `services/` để tải mô hình, thực hiện inference, và trả về kết quả (boxes, labels, scores).
- **Trả về response**: Trả về kết quả dưới dạng JSON, sử dụng schema Pydantic để đảm bảo định dạng chuẩn.

api/detect.py

```

1 from fastapi import APIRouter, UploadFile, File, HTTPException, Depends, Query
2 from fastapi.responses import JSONResponse
3 from typing import Optional
4

```

```

5 from app.services.detection_service import DetectionService
6 from app.models.schemas import DetectionResponse, ErrorResponse, DetectionRequest
7 from app.api.dependencies import get_token_header
8
9 router = APIRouter(
10     prefix="/detect",
11     tags=["detection"],
12     dependencies=[Depends(get_token_header)],
13     responses={401: {"description": "Unauthorized"}}
14 )
15
16 @router.post(
17     "/image",
18     response_model=DetectionResponse,
19     responses={400: {"model": ErrorResponse}, 500: {"model": ErrorResponse}}
20 )
21 async def detect_objects(
22     file: UploadFile = File(..., description="Image file to process"),
23     threshold: Optional[float] = Query(None, description="Detection confidence threshold")
24 ):
25     """
26     Detect fashion objects in an uploaded image.
27
28     - **file**: Image file (JPEG, PNG, etc.)
29     - **threshold**: Optional confidence threshold (default: 0.4)
30     """
31     # Validate file type
32     if not file.content_type.startswith('image/'):
33         raise HTTPException(
34             status_code=400,
35             detail="File must be an image (JPEG, PNG, etc.)"
36         )
37
38     # Read and process image
39     try:
40         image_bytes = await file.read()
41         result = DetectionService.detect_from_bytes(image_bytes, threshold)
42
43         if not result.success:
44             raise HTTPException(status_code=500, detail=result.error)
45
46         return result
47
48     except Exception as e:
49         raise HTTPException(
50             status_code=500,
51             detail=f"Error processing image: {str(e)}"
52         )

```

III.4. Schema Dữ liệu

Schema được định nghĩa trong `models/` sử dụng Pydantic để đảm bảo tính hợp lệ của dữ liệu. Ví dụ:

- **DetectionRequest:** Xác định cấu trúc của input, thường chỉ cần UploadFile cho ảnh.
- **DetectionResponse:** Xác định cấu trúc của output, ví dụ danh sách các item được phát hiện với thông tin label, score, và box.

models/schemas.py

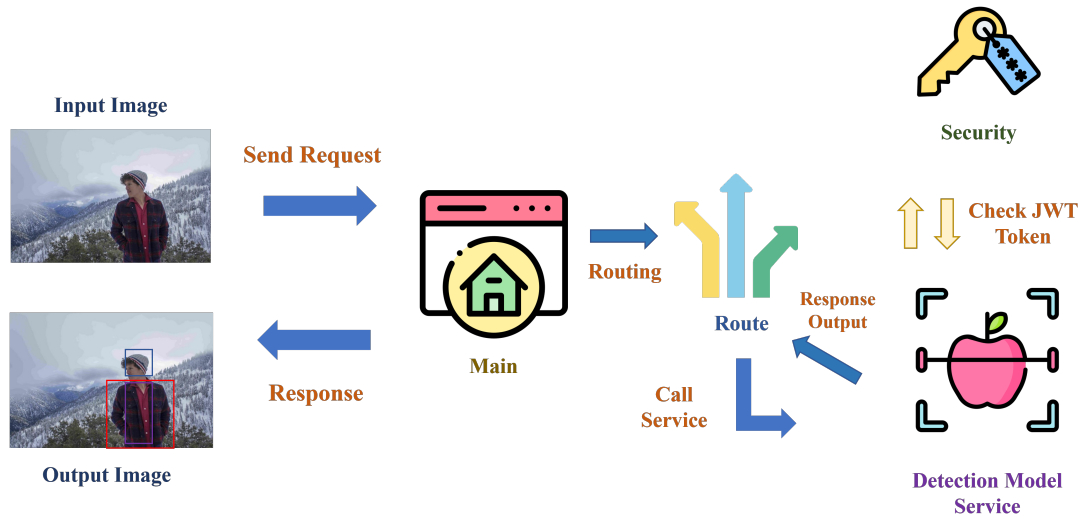
```

1 from pydantic import BaseModel, Field
2 from typing import List, Optional
3 from datetime import datetime
4
5 class BoundingBox(BaseModel):
6     xmin: float = Field(..., description="Left coordinate of bounding box")
7     ymin: float = Field(..., description="Top coordinate of bounding box")
8     xmax: float = Field(..., description="Right coordinate of bounding box")
9     ymax: float = Field(..., description="Bottom coordinate of bounding box")
10
11 class DetectionResult(BaseModel):
12     label: str = Field(..., description="Detected object class")
13     score: float = Field(..., description="Confidence score")
14     bounding_box: BoundingBox = Field(..., description="Bounding box coordinates")
15
16 class DetectionRequest(BaseModel):
17     threshold: Optional[float] = Field(None, description="Detection threshold")
18
19 class DetectionResponse(BaseModel):
20     success: bool = Field(..., description="Request success status")
21     detections: List[DetectionResult] = Field(..., description="List of detected
22                                             objects")
23     processing_time: float = Field(..., description="Time taken to process the image")
24     image_size: dict = Field(..., description="Original image dimensions")
25
26 class HealthResponse(BaseModel):
27     status: str = Field(..., description="Service status")
28     version: str = Field(..., description="API version")
29     model_loaded: bool = Field(..., description="Model loading status")
30     device: str = Field(..., description="Device used for inference")
31
32 class ErrorResponse(BaseModel):
33     success: bool = Field(False, description="Request success status")
34     error: str = Field(..., description="Error message")
35     details: Optional[str] = Field(None, description="Additional error details")

```

III.5. Flow Đọc hiểu Code

Luồng xử lý code trong project tuân theo quy trình từ nhận request đến trả response:

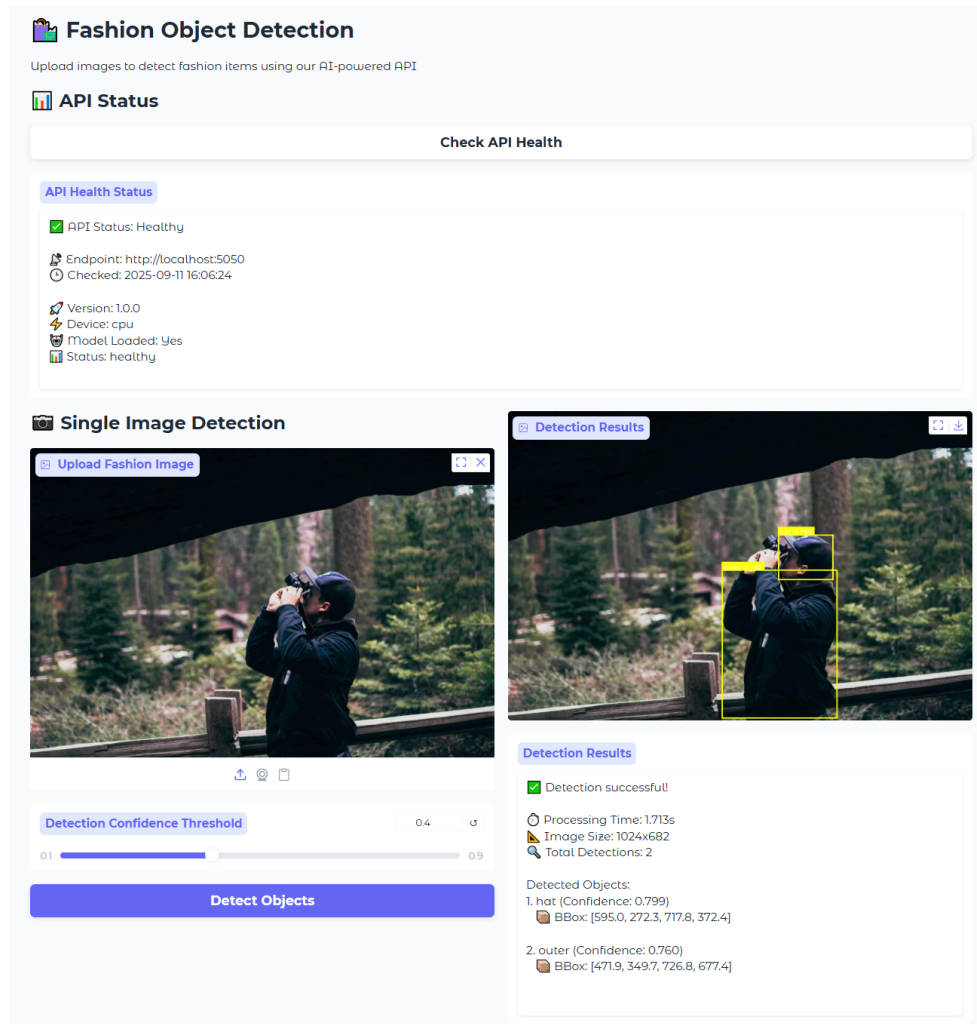


Hình 3: Flow xử lý của app khi nhận request tới response

1. **main.py**: Khởi tạo ứng dụng FastAPI, include router từ `api/`, và thiết lập middleware (như CORS hoặc authentication từ `core/`).
2. **Route (api/)**: Khi request đến (ví dụ: `POST /detect`), endpoint nhận file ảnh, validate schema (từ `models/`), và gọi hàm xử lý từ `services/`.
3. **Security (core/)**: Kiểm tra quyền truy cập của request, như xác thực user qua JWT Key, trước khi xử lý request.
4. **Services (services/)**: Thực hiện logic chính:
 - Tải mô hình từ Hugging Face (`yainage90/fashion-object-detection`).
 - Sử dụng PIL để xử lý ảnh đầu vào.
 - Chạy inference với mô hình, trả về `boxes`, `labels`, `scores`.
 - Xử lý kết quả để định dạng theo schema `DetectionResponse`.
5. **Response**: Kết quả từ `services/` được đóng gói thành JSON và trả về client. Nếu có lỗi (như file ảnh không hợp lệ), FastAPI raise `HTTPException`.

III.6. Giao diện Gradio

Mọi người hãy thực hiện cài đặt được hướng dẫn chi tiết ở repo **fashion-detection-app**, sau đó khởi chạy Gradio UI để thực hiện thử nghiệm detect ảnh theo batch hoặc từng ảnh một.



Hình 4: Giao diện tương tác với API endpoint triển khai bằng gradio

IV. Kết luận

FastAPI là một framework hiệu quả mà đơn giản, dễ triển khai. Đối với các AI reseacher hay AI engineer không quá giỏi software, FastAPI là một lựa chọn cực kỳ mạnh để chúng ta thực hiện serving model. Ngoài các phần đã được nói ở trong bài, mọi người có thể tìm hiểu thêm các khái niệm khác như kết nối database, testing API, logging, monitoring hay các kiến thức khác về security cho API. Từ đó phát triển được các ứng dụng tận dụng triệt để thế mạnh của mô hình mà mình xây dựng được trong quá trình nghiên cứu.